tensor4all development update

Contributors, tree methods, tenferro, and the kernel libraries

tensor4all developers

tensor4all meeting, 2026-10-27

Who contributed since September 22

Name	Where
Samuel Badr	tensor4all-rs
Simon Becker	tensor4all-rs: tree network storage (HDF5)
Lingrui Cheng	tensor4all-rs: tree patching, TreeRSI, tree reliability
Lukas Devos	tcontract-rs: the original contraction kernels
Ken Inayoshi	tensor4all-rs: performance, quantics transform precision
Martin Mikkelsen	tlinalg-rs, t4a-faer, tensor4all-rs, tenferro-rs
Marc K. Ritter	AlternatingCrossInterpolation.jl
Rihito Sakurai	tensor4all-rs: TCI pivot search fixes
Hiroshi Shinaoka	most repositories
Satoshi Terasaki	tenferro-rs, tenferro-benchmark, CI bot
Ryo Watanabe	t4a-faer

Commit authors and co-authors on the default branches of tensor4all repositories, 2026-09-22 to 2026-10-27, alphabetical. Also upstream faer authors (via sync) and AI coding agents under human review.

Activity since September 22

11
people committed

33
repositories touched

15
new public repositories

Top: tree methods in tensor4all-rs

Middle: tenferro engine and a new GPU path

Bottom: kernels split out of tenferro

New repositories include tcontract-rs, cpueinsum-rs, tlinalg-rs, the faer fork, t4a-cuda-kernels, t4a-gpu-backend, threadteam-rs and their benchmark repositories.

The stack

Frontends	tensor4all-rs · Tensor4all.jl · tensor4all-py
Algorithms	tensor trains, trees, TCI, patching, quantics
Tensor runtime	tenferro-rs: tensors, autodiff, CPU and GPU
Kernel providers	**cpueinsum-rs · tcontract-rs · tlinalg-rs**
Foundation	**strided-rs · t4a-faer**

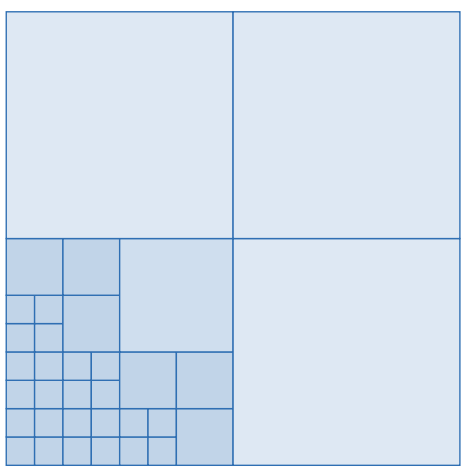
Top: what users call **Middle: the runtime** **Bottom: the engine**

Interactive version: tensor4all.org/tensor4all-meta/stack/

Top: adaptive patching on trees

tensor4all-rs

Published for tensor trains; **Lingrui Cheng extended it to trees.**



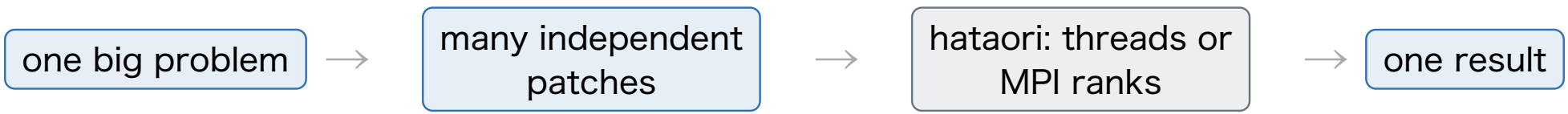
- Split the domain **only where the function is hard**
- Every patch is a small tree with the same shape
- Works for chains and branched trees alike

On one test the patched run was slower and less accurate than a single network. When patching pays off is still open.

G. Grosso, M. K. Ritter, S. Rohshap, S. Badr, A. Kauch, M. Wallerberger, J. von Delft, H. Shinaoka, Adaptive patching for tensor train computations, arXiv:2602.22372. Trees: tensor4all-rs PR 900

Top: parallel patching with hataori

tensor4all-rs



- Patches are independent, so they can run on different cores or nodes
- **hataori-rs** hands out the work; each worker uses its own tenferro engine
- Chains and trees share one implementation; the chain-only crate was removed

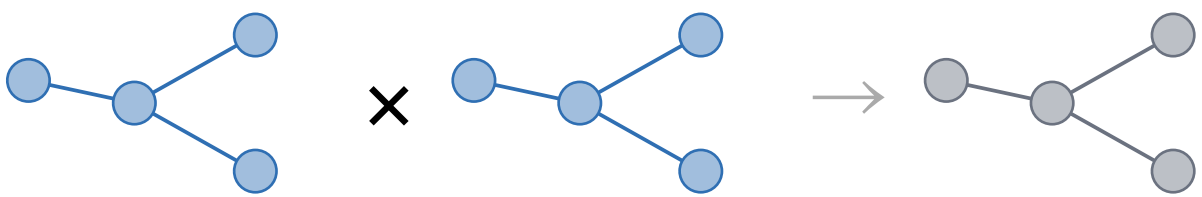
Tested for correctness on 1 and 2 MPI ranks. Scaling not measured yet.

tensor4all-rs PRs 1005, 1009 hataori-rs PRs 25, 27

Top: TreeRSI

tensor4all-rs

Published for tensor trains; **Lingrui Cheng extended it to trees.**



- Element-by-element product of two trees, compressed in one pass
- Checked against the original authors' code, with the same random numbers

Z. Meng, Y. Khoo, J. Li, E. M. Stoudenmire, Recursive sketched interpolation: efficient Hadamard products of tensor trains, arXiv:2602.17974.

Top: other fixes in tensor4all-rs

tensor4all-rs

Convergence reports

Lingrui Cheng

Says “rank limit reached” instead of a false “converged”.

TCI pivots

Rihito Sakurai

Pivot search no longer stops at zero residuals or forgets good moves.

Speed

Ken Inayoshi, Martin Mikkelsen

Fewer copies in core loops; column-major grids in TCI.

Storage

Simon Becker

Tree networks saved to HDF5 keep their canonical form.

Full details of the tree work: slides/tensor4all20261027-lingrui-cheng.pdf

Top: remaining work

tensor4all-rs

Patched products

biggest gap

Multiply two patched operators (MPO-MPO) end to end.

Error checks

cheaper and safer

Catch local errors that sampling misses, at lower cost; fix remaining TreeACI convergence issues.

Scaling

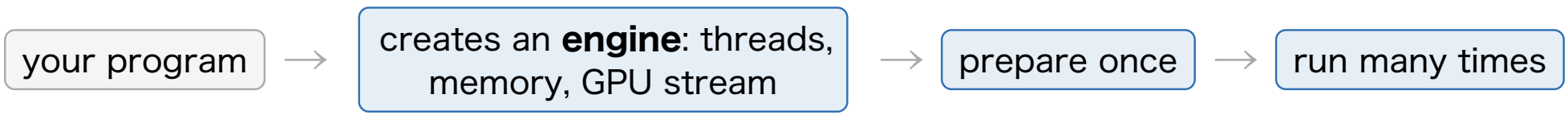
not measured yet

Run the MPI patching at scale and measure.

Source: Lingrui Cheng's slides, "Remaining work".

Middle: tenferro execution engine

tenferro-rs

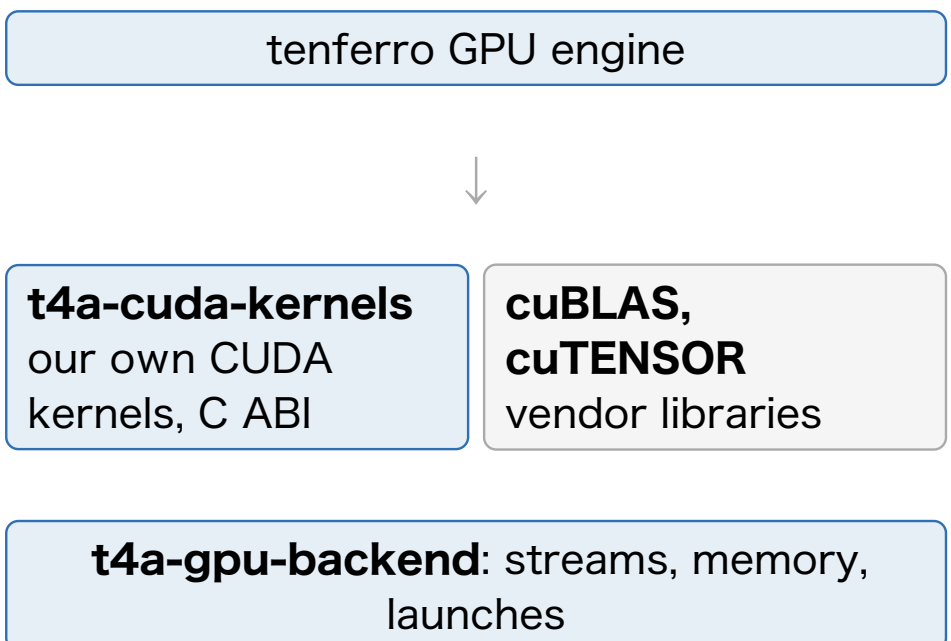


- The engine is an ordinary value: you create it and pass it to each call
- Two engines can run side by side, for example one per MPI rank or per patch
- Small operations stay on the calling thread

This is a breaking change; the old session API was removed.

Middle: new CUDA backend

tenferro-rs

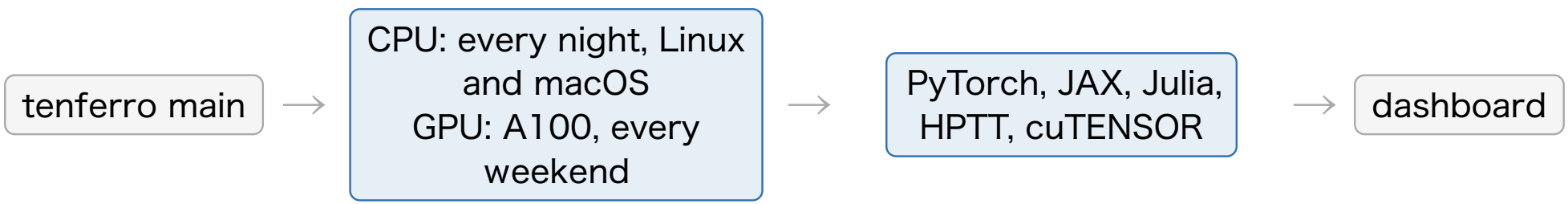


- Replaces the earlier CubeCL-based CUDA path
- The kernels use no tenferro types, so other CUDA codes can call them
- Matrix products via cuBLAS, contractions via cuTENSOR
- Migration is in progress, piece by piece; tested on an A100

tenferro-rs 2208, PRs 2250 to 2263 t4a-cuda-kernels; t4a-gpu-backend.

Middle: tenferro-benchmark

tenferro-rs

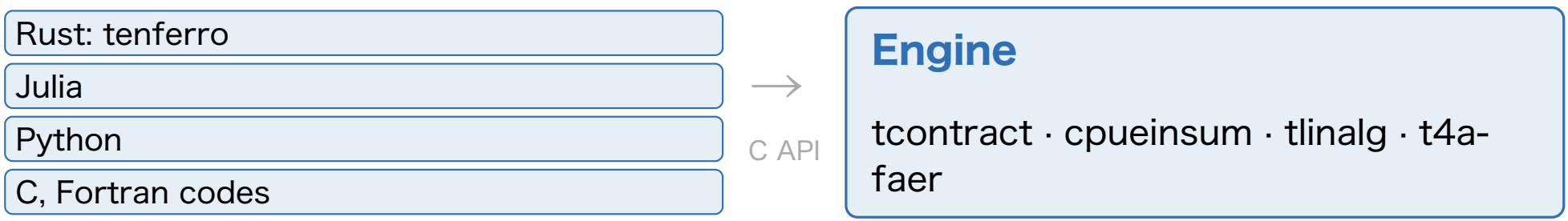


- Runs start automatically from GitHub Actions; the GPU runs use rented A100s
- Raw data and settings for every run are kept in the repository
- Example, A100, a tensor network contraction (f32): tenferro 49 ms, PyTorch 111 ms
- Dashboard: tensor4all.org/tenferro-benchmark

github.com/tensor4all/tenferro-benchmark; example from result/nvidia-gpu/gpu/tensornetwork.md, 2026-10-24, tenferro 06ac247. Most changes since September 22 by Satoshi Terasaki.

Bottom: why split out the kernels

engine



Started with **Lukas Devos** (CCQ, developer of TensorKit.jl) and his Rust prototype.

- **Julia:** call precompiled kernels, less precompile time (future)
- Fast kernels are too much work for one group to maintain
- **We want to maintain them together with other groups**

Bottom: four libraries

engine

tcontract-rs

multiply two tensors

From **Lukas Devos's prototype**. C API follows **TAPP**.

cpueinsum-rs

multiply many tensors

Cut out of tenferro. Lower per-step overhead.

tlinalg-rs

SVD, QR, LU, eigen

Thin layer over t4a-faer. **Batch** API like PyTorch.

t4a-faer

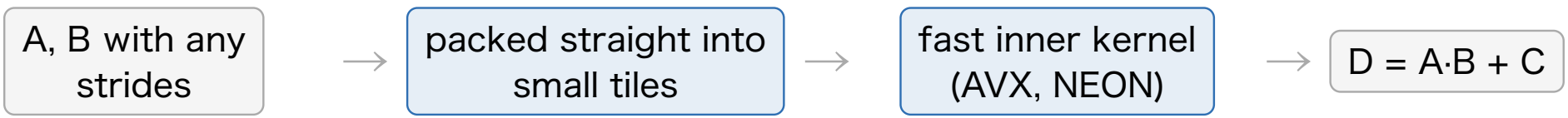
fork of faer

New parallel model, bug fixes, **AI-driven** optimization.

All four now on crates.io and used by tenferro.

Bottom: tcontract-rs

engine

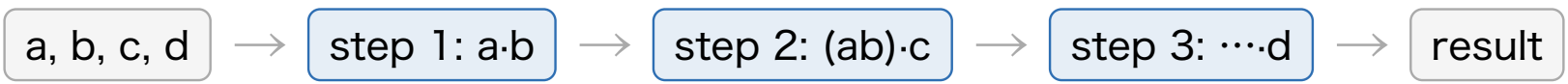


- No transpose or copy of the inputs first
- f32, f64 and complex; batch indices, traces, conjugation
- C API follows **TAPP**, a community standard for tensor products
- One shared library, `libtcontract_capi`, for any language with a C FFI

TAPP: arXiv:2601.07827. tcontract-rs README.

Bottom: cpueinsum-rs

engine



- **You** choose the order; cpueinsum plans every step once
- One reusable scratch buffer for all the intermediates
- Not included: order search, `"ij,jk->ik"` strings, GPUs
- Usable without tenferro

Bottom: tlinalg-rs, batched linear algebra

engine

Like PyTorch, tlinalg takes a **batch**: many matrices of the same shape in one call.

With 4 threads there are two ways to run it:

1. Give each thread its own share of the batch, a contiguous block of matrices.
2. Factorize the matrices one by one, each with all 4 threads.

For small matrices the threads in the second way spend most of their time coordinating, so **the first way is faster**. By default tlinalg picks the way itself; a caller can also fix it.

tlinalg-rs 0.6. The default policy estimates each matrix's work and splits the batch across threads only when that repays the cost of starting them.

Bottom: t4a-faer

engine

Fork of faer

dense linear algebra in Rust

We use it for fast experiments, mostly run by AI agents.

What we changed

Parallel model, bug fixes, faster QR and LU paths.

Experiment log

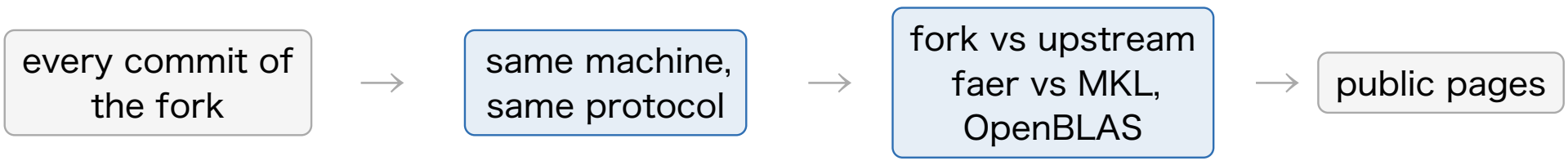
PATCHSET.md

Each change is recorded as kept or dropped, with the reason.

faer by Sarah Quiñones. Our fork: github.com/tensor4all/faer, crates.io t4a-faer.

Bottom: faer-benchmark

engine

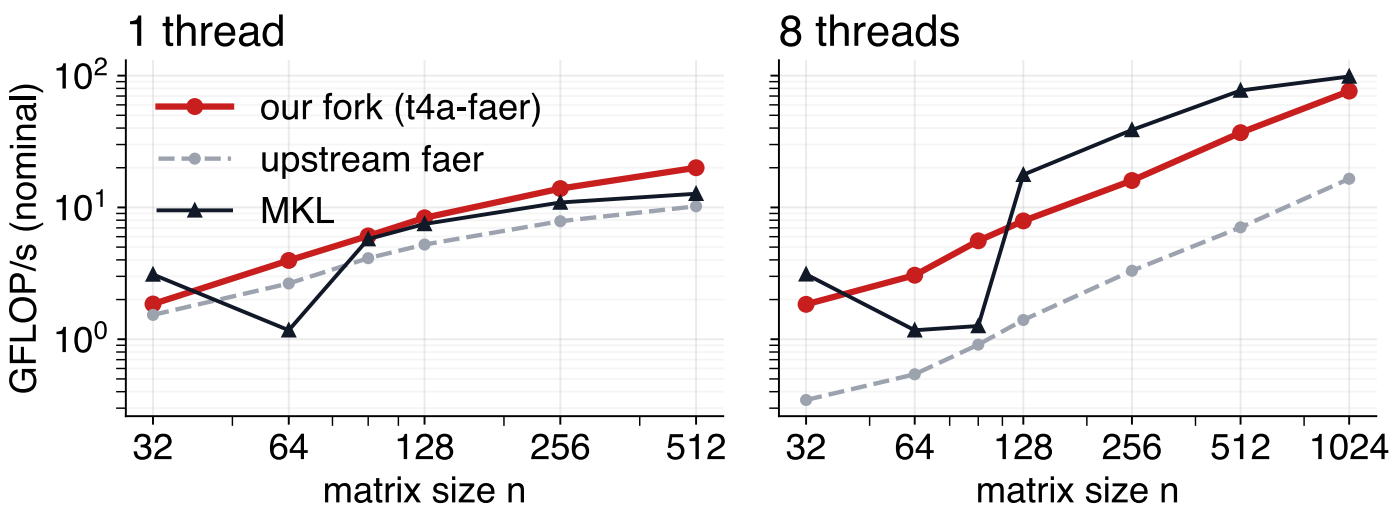


- Nine families: GEMM, LU, Cholesky, QR, SVD, eigenvalues, …
- Each result is tied to a commit and a machine; results from older commits are marked
- An unchanged control row checks the machine; if it moves, the result is not accepted
- Raw data, scripts and every run's settings are in the repository

github.com/tensor4all/faer-benchmark. Sibling repositories: tlinalg-benchmark, tcontract-benchmark, cpueinsum-benchmark, tenferro-benchmark.

Bottom: QR factorization, 1 vs 8 threads

engine



1 thread: our fork beats upstream faer and MKL.

8 threads: MKL still leads. **Parallel scaling is the next target.**

faer-benchmark, AMD Zen 5, f64, nominal GFLOP/s ($4n^3/3$); fork 50c2b98 (n = 1024: a5e6b7d) vs upstream faer 0.24.4. QR with an explicit Q shows the same trend. Provisional: development host.

Regular benchmarking

AI agents write most of this code.

Public, repeatable benchmarks let others check the results.

- tenferro-benchmark already runs on a schedule: CPU every night, A100 every weekend
- faer-benchmark runs on a development host for now; a dedicated machine will rerun it regularly
- We will show the trends at these meetings

Discussion

- **Top:** which tree workloads should we test with real users?
- **Middle:** who needs the GPU path first?
- **Bottom:** who else wants to share the kernels with us?
- **t4a-faer:** send our fixes back to upstream faer?

References

- Tree methods in detail (Lingrui Cheng): `slides/tensor4all20261027-lingrui-cheng.pdf` and its notes
- Adaptive patching: G. Grosso et al., arXiv:2602.22372
- RSI: Z. Meng, Y. Khoo, J. Li, E. M. Stoudenmire, arXiv:2602.17974
- TAPP: arXiv:2601.07827
- Stack map: tensor4all.org/tensor4all-meta/stack/
- Code: github.com/tensor4all (tensor4all-rs, tenferro-rs, tcontract-rs, cpueinsum-rs, tlinalg-rs, faer)